SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5 TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I Thanveer T (192424107) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A
CASE STUDY — ANSWERS

Question 1: Weak Password Hashing in Web Application

Scenario: *A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.*

Analysis of the Vulnerability

The vulnerability in this scenario stems from two independent weaknesses that, together, make password recovery almost trivial for an attacker once the database is exposed. SHA-1 was designed as a fast, general-purpose cryptographic hash function, primarily intended for integrity checking rather than password protection. Its speed, which is desirable for checksums, becomes a liability for password storage because it allows an attacker to compute billions of hash guesses per second using modern GPUs. In addition, SHA-1 has known collision weaknesses, which further erodes confidence in its use for any security-critical application.

The absence of a salt compounds this problem severely. A salt is a unique, random value appended to each password before hashing, ensuring that identical passwords produce different hash outputs across users. Without salting, two users who choose the same password will have identical hash values, and more importantly, an attacker can pre-compute a large dictionary of hash values for common passwords — known as a rainbow table — and instantly reverse-lookup any matching hash in the stolen database. This is precisely why the breach in the scenario led to rapid mass cracking of passwords rather than requiring individual brute-force attempts.

- Fast hashing (SHA-1) enables high-speed brute-force and dictionary attacks.
- No salting allows reuse of precomputed rainbow tables across the entire user base.
- Identical passwords produce identical hashes, exposing patterns of password reuse.
- SHA-1's cryptographic weaknesses further reduce attacker effort and increase risk of collisions.

Recommended Secure Hashing Strategy

The organisation should migrate to a password hashing algorithm that is specifically designed to be slow and computationally expensive, such as bcrypt, scrypt, or Argon2 (the current OWASP-recommended default). These algorithms incorporate a configurable work factor, allowing the computational cost to be increased over time as hardware improves, and they generate and store a unique salt automatically for every password.

- Use Argon2id (or bcrypt/scrypt as alternatives) with a unique, randomly generated salt per password.
- Apply a high iteration/work-factor count to slow down brute-force attempts, tuned to acceptable login latency.
- Add a server-side secret (pepper) stored outside the database to provide defence-in-depth if the database alone is compromised.
- Enforce strong password policies and multi-factor authentication to reduce reliance on password strength alone.
- Periodically rehash stored passwords with updated parameters as computing power increases.

Justification: Argon2id is memory-hard, meaning it deliberately consumes large amounts of RAM in addition to CPU time, which severely limits the effectiveness of parallel cracking on GPUs and ASICs — the exact attack vector that succeeded against the unsalted SHA-1 implementation. Combined with per-user salts, even identical

passwords will yield completely different stored hashes, eliminating the value of precomputed tables and forcing attackers back to slow, per-account brute-force attempts.

Real-World Relevance

This scenario mirrors several well-documented breaches — including the 2012 LinkedIn incident — where unsalted SHA-1 hashes were exposed and the vast majority of passwords were recovered within days using publicly available rainbow tables and GPU-based cracking rigs. These real incidents were a major factor in industry-wide migration toward OWASP-recommended algorithms such as bcrypt and Argon2, and in regulators and auditors now explicitly flagging unsalted fast hashes as a critical finding during security assessments.

Conclusion

This case demonstrates that hashing algorithm choice and salting practice are not interchangeable implementation details but core security controls in their own right. A password storage design should always be evaluated against modern, purpose-built standards rather than general-purpose hash functions, and should be revisited periodically as computational capability and best practice evolve.

Question 2: Digital Signature Compromise in Financial System

> ***Scenario:*** *A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.*

Impact on Authentication and Non-Repudiation

Digital signatures rely on the fundamental assumption that a private key is known only to its legitimate owner. Authentication in this scheme works because a transaction signed with a specific private key can only be verified successfully using the corresponding public key, allowing the recipient to confirm the signer's identity. Once the private key is compromised, this assumption collapses: the attacker can produce signatures that are mathematically indistinguishable from those of the legitimate user, so the system will authenticate fraudulent transactions as genuine.

Non-repudiation is affected even more severely. Non-repudiation is the property that prevents a signer from later denying that they authorised a transaction, since only they possess the private key. When the key is stolen, this guarantee is broken in both directions — the legitimate user can plausibly deny fraudulent transactions performed by the attacker, and the institution loses the ability to prove, after the fact, which party actually approved a given transaction. This creates significant legal and financial exposure, since disputed transactions can no longer be conclusively attributed.

- Authentication is defeated because the attacker can generate valid signatures indistinguishable from the real user's.
- Non-repudiation collapses, as the legitimate user can deny transactions they did not perform.
- Trust in the institution's transaction approval workflow is undermined, along with potential regulatory and audit consequences.
- Downstream systems that rely on the signature for authorization will process fraudulent transactions as legitimate.

Mitigation Strategies

Protecting private keys and limiting the blast radius of a compromise requires a combination of secure key storage, procedural controls, and rapid response mechanisms.

- Store private keys in Hardware Security Modules (HSMs) or smart cards so the key material never leaves tamper-resistant hardware.
- Require multi-factor authentication before any signing operation is authorised, so a stolen key alone is insufficient.
- Implement transaction-value thresholds requiring multi-party (dual-control) signatures for high-value transfers.
- Maintain short-lived certificates and enable rapid key revocation via Certificate Revocation Lists (CRLs) or OCSP the moment compromise is suspected.
- Monitor for anomalous transaction patterns using behavioural analytics to flag suspicious activity in real time.
- Educate users on key-handling hygiene and enforce secure key generation, storage, and backup procedures.
- Maintain a documented key-compromise response plan covering immediate revocation, customer notification, and forensic review timelines.

Together, these controls shift the security model from relying solely on a single private key to a layered defence, so that even if one factor is compromised, unauthorized transactions can be detected, contained, or blocked before completion.

Regulatory and Legal Considerations

Financial regulators generally require institutions to demonstrate strong key-management controls under frameworks such as PCI DSS and applicable national digital-signature laws. Following a private-key compromise, the institution must typically notify affected customers, cooperate with forensic investigation, and in many jurisdictions bear the burden of proving that a disputed transaction was not authorised through institutional negligence, which makes preventive key-protection controls a legal as well as a technical necessity.

Conclusion

The incident highlights that private-key security is the foundation on which both authentication and non-repudiation rest in any digital-signature scheme. Financial institutions must therefore treat key custody as a first-class security control, backed by hardware protection, procedural checks, and rapid revocation capability, rather than assuming that possession of a private key is, by itself, a permanent and unassailable guarantee of identity.

Question 3: Kerberos Authentication Failure in Distributed System

Scenario: *A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.*

Impact of Time Synchronization on Kerberos

Kerberos is a ticket-based authentication protocol that depends heavily on timestamps to prevent replay attacks. Each ticket issued by the Key Distribution Centre (KDC) contains a timestamp and a limited validity window (typically a few minutes), and the client and server compare this timestamp against their own local clocks to decide whether the ticket is fresh. This design assumes that all participating machines share a reasonably consistent view of time.

When clocks drift apart beyond the protocol's allowed clock-skew tolerance (commonly around five minutes by default), tickets that are actually valid may be rejected as expired or as suspected replay attempts, since the receiving server cannot distinguish a genuinely stale ticket from one affected by clock drift. This produces exactly the symptom described in the scenario: frequent, seemingly random authentication failures that are not caused by incorrect credentials but by inconsistent system clocks across the distributed environment.

- Tickets outside the permitted clock-skew window are rejected, even if the credentials themselves are valid.
- Excessive clock drift can be misinterpreted as a replay attack, causing legitimate requests to fail authentication.
- Session tickets may expire prematurely or remain valid longer than intended, weakening the security guarantees Kerberos is meant to provide.
- Failures tend to appear intermittent and difficult to diagnose, since they depend on the varying degree of drift on each host.

Recommended Solutions

The core fix is to ensure that every host participating in the Kerberos realm maintains an accurate and consistent clock, supported by monitoring to catch drift before it causes failures.

- Deploy a centralized, authoritative Network Time Protocol (NTP) or Precision Time Protocol (PTP) server that all domain controllers, application servers, and clients synchronize against.
- Configure all systems to synchronize time from the same hierarchy of trusted time sources to avoid divergence between subsystems.
- Set an appropriate maximum clock-skew tolerance in the Kerberos configuration, balancing security against the practical drift observed in the environment.
- Monitor time synchronization status continuously and generate alerts when a host's drift approaches the allowed tolerance.
- Ensure virtualized or containerized hosts synchronize with the host clock correctly, since virtualization is a common hidden source of clock drift.
- Document a standard operating procedure for diagnosing authentication failures that begins with a clock-skew check before escalating to credential or network investigation.

With reliable time synchronization in place, tickets will be evaluated consistently across the distributed system, eliminating false authentication failures while still preserving the replay-attack protection that timestamp validation is designed to provide.

Real-World Relevance

Large enterprise environments running Active Directory, which relies on Kerberos as its default authentication protocol, commonly encounter exactly this class of issue when virtual machines are cloned or resumed from snapshots, since hypervisor pauses can silently introduce minutes of clock drift. Microsoft's own guidance for Active Directory environments explicitly documents the five-minute default skew tolerance and recommends the domain controller holding the PDC Emulator role act as the authoritative time source for the entire domain.

Conclusion

This case shows that Kerberos security and availability both depend on infrastructure that is often treated as a mere operational detail. Reliable time synchronization should be designed, monitored, and maintained with the same rigor as the authentication protocol itself, since a breakdown in one silently undermines the other.

Question 4: Certificate Authority Breach in PKI System

Scenario: *A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.*

Impact on Trust and Secure Communication

Public Key Infrastructure (PKI) is built on a chain of trust that ultimately traces back to a small set of root and intermediate Certificate Authorities. Browsers, operating systems, and applications trust any certificate signed by a CA in their trust store, without independently verifying the identity behind each certificate. When a CA is compromised, an attacker can issue fraudulent X.509 certificates for any domain, including high-value targets such as banking or email providers, and these certificates will be accepted as fully legitimate by any relying system.

The consequences extend well beyond the specific domains targeted. Once a CA compromise is discovered, every certificate ever issued by that CA becomes suspect, since there is no way to be certain which certificates were issued legitimately and which were fraudulently generated. This can enable man-in-the-middle attacks, phishing sites that present valid-looking HTTPS certificates, and interception of encrypted traffic, all while presenting no visible warning to end users.

- Attackers can impersonate trusted domains, enabling undetected man-in-the-middle attacks on encrypted (HTTPS/TLS) traffic.
- End users receive no browser warning, since the fraudulent certificate chains to a trusted root.
- The entire trust hierarchy under the compromised CA is called into question, not just the fraudulently issued certificates.
- Recovery requires revoking and reissuing potentially large numbers of certificates, causing widespread operational disruption.

Detection and Mitigation Mechanisms

A layered strategy is needed both to detect fraudulent certificates quickly and to limit the damage a compromised CA can cause across the wider ecosystem.

- Adopt Certificate Transparency (CT) logs so every issued certificate is publicly logged and can be audited by domain owners for unauthorized issuance.
- Use CAA (Certification Authority Authorization) DNS records to restrict which CAs are permitted to issue certificates for a given domain.
- Deploy real-time Certificate Revocation List (CRL) and OCSP stapling checks so relying parties can detect and reject revoked certificates promptly.
- Enforce HSM-protected root and intermediate keys with strict access controls and offline storage for root CA keys to reduce compromise likelihood.
- Apply certificate pinning for critical, high-value applications to reduce reliance on the full public CA ecosystem.

- Establish an incident response plan for CA compromise, including rapid mass revocation and public disclosure procedures.
- Require multi-party approval (four-eyes principle) within the CA's own issuance workflow so that a single compromised credential cannot alone trigger fraudulent issuance.

These mechanisms do not eliminate the possibility of a CA compromise, but they substantially reduce detection time and limit the window during which fraudulent certificates can be exploited before being identified and revoked.

Real-World Relevance

This scenario reflects real incidents such as the 2011 DigiNotar breach, where attackers issued fraudulent certificates for major domains and the resulting loss of confidence led browser vendors to permanently distrust the CA, effectively ending its operations. That event was also a key driver behind the industry's later adoption of Certificate Transparency as a mandatory requirement for publicly trusted certificates.

Conclusion

A CA breach demonstrates that trust in PKI is only as strong as the weakest CA in the chain that a system is willing to accept. Transparency, monitoring, and issuance restriction mechanisms are therefore essential complements to cryptographic strength, since they address the operational and organisational risks that pure cryptography cannot solve on its own.

Question 5: Integrity Verification in Cloud Data Storage

> ***Scenario:*** *A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.*

Analysis of the Integrity Failure

Hashing alone verifies that data has not changed since a hash was recorded, but it does not, by itself, verify who produced that hash or protect the hash value itself from tampering. If the provider computes a hash of each file and stores that hash alongside the file — without any additional protection — an attacker who can modify the file can simply recompute the hash of the tampered file and overwrite the stored value. In this case, integrity verification will always report success even though the file's content has been altered, because the check compares the modified file to a matching, equally modified hash.

Other common causes of improper hash validation include using outdated or weak hash algorithms susceptible to collision attacks, failing to verify hashes at every stage of storage and retrieval (rather than only at upload time), and not authenticating the source of the hash itself. Together, these gaps mean the integrity mechanism is present in name but does not actually provide a trustworthy guarantee against tampering, which matches the tampering incidents reported by users.

- Storing the hash alongside the file with no independent protection allows both to be modified together undetected.
- Verifying integrity only at upload, and not on subsequent access or storage cycles, misses tampering that occurs later.
- Use of weak or collision-prone hash algorithms undermines the reliability of the integrity check itself.
- No cryptographic binding exists between the hash and a verified, trusted source, so authenticity of the hash cannot be confirmed.

Recommended Integrity Verification Mechanism

To provide a robust guarantee, integrity verification must ensure both that the data is unaltered and that the verification value itself cannot be forged or silently replaced by an attacker with write access to storage.

- Use HMAC (Hash-based Message Authentication Code) with a secret key held separately from the storage system, so an attacker without the key cannot regenerate a valid tag for tampered data.
- Digitally sign file hashes using the data owner's private key, providing both integrity and non-repudiation for stored content.
- Employ Merkle trees for large datasets, allowing efficient verification of individual blocks while any single change is detectable from a small root hash.
- Store integrity metadata (hashes/signatures) in a separate, access-controlled, and ideally immutable store, distinct from the data itself.
- Perform periodic automated integrity audits comparing stored data against verified hashes/signatures, not only at upload or download time.
- Adopt current, collision-resistant hash algorithms such as SHA-256 or SHA-3 rather than deprecated functions.
- Log and version integrity checks so that any discrepancy can be traced to a specific point in time and a specific storage operation.

Justification: HMACs and digital signatures anchor the integrity check to a secret or private key that the attacker does not possess, so tampering with the data can no longer be concealed by simply recomputing a plain hash. Combined with periodic audits and separated, access-controlled storage of integrity metadata, this approach closes the gap that allowed undetected tampering in the original implementation.

Real-World Relevance

Cloud providers increasingly document integrity guarantees explicitly, for example through checksums returned by object storage APIs and optional customer-managed HMAC or digital-signature verification layers. Enterprises handling regulated data typically layer their own signature-based integrity checks on top of the provider's native mechanism, precisely because relying solely on a provider-generated hash offers no protection if the provider's own storage or hash-generation process is itself compromised or misconfigured.

Conclusion

Effective integrity verification requires more than the presence of a hash value; it requires that the hash (or signature) itself be protected from the same threat it is meant to detect. Cloud providers should treat integrity metadata as a security-critical asset in its own right, governed by access control, key management, and independent auditing.

Overall Summary and Key Takeaways

Across all five cases, a common thread emerges: cryptographic mechanisms — hashing, digital signatures, Kerberos tickets, and X.509 certificates — are only as trustworthy as the surrounding implementation, key-management, and operational practices that support them. In each scenario, the underlying cryptographic primitive was not fundamentally broken; rather, a gap in configuration, process, or infrastructure allowed the primitive's guarantees to be undermined or bypassed entirely.

- Password hashing must use slow, memory-hard, salted algorithms (e.g. Argon2id) rather than fast general-purpose hashes such as unsalted SHA-1.
- Digital signature security depends entirely on private-key custody; hardware-backed storage and multi-factor controls are essential to preserving authentication and non-repudiation.
- Kerberos authentication requires accurate, monitored time synchronization across all hosts, since ticket validity is timestamp-dependent.
- PKI trust is systemic: a single compromised CA can undermine confidence across every certificate it has issued, making transparency and monitoring essential.
- Integrity verification must protect the hash or signature itself, not just compute it, or the mechanism provides only an illusion of assurance.

Taken together, these cases reinforce that cryptography and network security is as much a discipline of sound engineering practice, key management, and operational discipline as it is a discipline of mathematics — a system is only as secure as its weakest implementation detail.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand